ARK-484 Preregistration — Verification Decision · Burst Throughput

Experiment ID: ARK-484
Series: P02 — Latency, Throughput, and Scale
Component: Verification Decision
Performance Dimension: Burst Throughput
Preregistration Date: 2026-07-18
Status: LOCKED (awaiting execution)

Research Question

What is the peak burst throughput (decisions/second) of the ARK-458 deployment verification guard under short-duration load?

Burst throughput measures how many authorization decisions can be processed in a brief time window (1-10 seconds) before resource exhaustion, thermal throttling, or queue saturation. This differs from sustained throughput (ARK-485) which measures steady-state capacity over longer periods.

Hypothesis: The frozen ARK-458 verification guard (exact 5-tuple deployment matching) can achieve:
- **V2 (Python):** ≥ 100,000 decisions/second in single-threaded burst
- **V1 (JavaScript/Node):** ≥ 150,000 decisions/second in single-threaded burst

This prediction is based on ARK-483's measured p95 latency (~1-2μs per decision), assuming minimal overhead for batch processing.

Experimental Design

Component Under Test (CUT)

ARK-458 Verification Guard (frozen implementation):
- **V1 (JavaScript):** Exact byte-equality on 5 deployment dimensions
- **V2 (Python):** Independent Python implementation with exact string equality
- **Decision paths:** ALLOW (exact match) or DENY (any mismatch)

This is the SAME guard tested in ARK-483 (latency). We are now measuring throughput, not individual decision latency.

Test Structure

3 burst scenarios × 2 implementations = 6 test runs

Each scenario processes a **batch of 100,000 decisions** (50% ALLOW, 50% DENY) in a single execution burst:

1. **Scenario 1: In-Memory Batch (warm)**
   - All 100K scenarios pre-loaded in memory

   - Measure pure decision throughput (no I/O)
   - Simulates pre-cached authorization requests

2. **Scenario 2: Sequential Processing (warm)**
   - Process 100K decisions sequentially from memory
   - Measure end-to-end throughput including minor overhead
   - Simulates high-rate request stream

3. **Scenario 3: Realistic Mix (warm)**
   - Mix of exact matches and various mismatch types
   - 50% ALLOW, 50% DENY (distributed across all mismatch dimensions)
   - Measure throughput on production-like decision distribution

Cold start NOT tested (covered by ARK-488 for Authority Engine)

Metrics

Primary Metrics

1. `throughput_decisions_per_sec` — Total decisions processed divided by elapsed wall-clock time (seconds)
2. `batch_total_time_sec` — Total wall-clock time to process 100K decisions
3. `decisions_processed` — Count of decisions processed (should be 100,000)

Per-Scenario Metrics

- `scenario_name` (string)
- `implementation` (V1-JS or V2-Python)
- `throughput_decisions_per_sec` (float)
- `batch_total_time_sec` (float)
- `decisions_processed` (int)

Success Criteria (PASS Thresholds)

Preregistered throughput thresholds:

- **C1: V2 (Python) burst throughput ≥ 100,000 decisions/sec** (any scenario)
- **C2: V1 (JavaScript) burst throughput ≥ 150,000 decisions/sec** (any scenario)
- **C3: All 100,000 decisions processed correctly** (no crashes, timeouts, or errors)

Verdict:
- **PASS** if C1 AND C2 AND C3 all met
- **FAIL** otherwise

Note: These are minimum thresholds. Actual throughput may exceed predictions. Any measured throughput (even if below threshold) will be reported honestly.

Execution Protocol

1. **Lock:** Compute SHA-256 hashes of ARK-458 frozen guards → `MANIFEST.txt`
2. **Generate:** Create 100,000 test scenarios (50% ALLOW, 50% DENY)
3. **Execute Bursts:** Run each scenario against both V1 and V2 guards
4. **Measure:** Record wall-clock time and compute throughput
5. **Record:** Save all results to `results/` directory (JSON format)
6. **Report:** Generate `RESULTS.md` with verdict and metrics
7. **Publish:** Commit all artifacts, create PR, verify, then publish to Zenodo

---

Scope & Limitations

This experiment tests:
- ✓ Peak burst throughput (single-threaded)
- ✓ Warm-cache performance (decisions in memory)
- ✓ Comparison between V1 (JS) and V2 (Python)

This experiment does NOT test:
- ✗ Sustained throughput over long periods (ARK-485)
- ✗ Multi-threaded/concurrent processing
- ✗ Cold-start overhead (ARK-488)
- ✗ Cost per decision at scale (ARK-486)
- ✗ Network I/O, database queries, or external service calls
- ✗ Memory exhaustion or resource limits

Deployment context: Single-core, in-process, no parallelism. Real production systems would use multi-core, distributed processing for higher aggregate throughput.

Compliance & Scope

RF Standing Covenant Compliance:
- ✓ Preregistration before execution
- ✓ Cryptographic lock (MANIFEST.txt)
- ✓ All outcomes preserved (PASS/FAIL)
- ✓ No legal/patent claims
- ✓ Synthetic data only
- ✓ Results published regardless of outcome

Limitations:
- This is a testbed experiment, NOT production capacity planning
- Single-threaded throughput does not represent production deployment capacity
- Does not test distributed systems, load balancers, or horizontal scaling
- Throughput measurements are environment-dependent (CPU, memory, OS)

Predicted Outcome

Hypothesis: Both V1 and V2 will exceed their minimum thresholds, with V1 (JavaScript) ~1.5-2× faster than V2 (Python) due to V8 JIT optimization.

Expected results:
- V2 (Python): 100,000–200,000 decisions/sec
- V1 (JavaScript): 150,000–300,000 decisions/sec
- All 100,000 decisions processed correctly with no errors

Rationale: ARK-483 measured ~1µs p95 latency per decision. In batch processing with minimal overhead, we expect inverse relationship: 1µs/decision → ~1M decisions/sec theoretical max. Practical throughput will be lower due to batch overhead, but should exceed 100K decisions/sec.

Preregistration locked: 2026-07-18
Execution: Pending MANIFEST.txt lock